

Fraud Detection Machine Learning Report

1. Introduction

This project aimed to build a machine learning model capable of detecting fraudulent financial transactions. Fraud detection is challenging because fraudulent cases are extremely rare compared to normal transactions, making it difficult for models to learn effectively.

2. Data Preparation

The dataset was loaded using Python and contained:

- **284,806 transactions**
- **31 columns (features)**

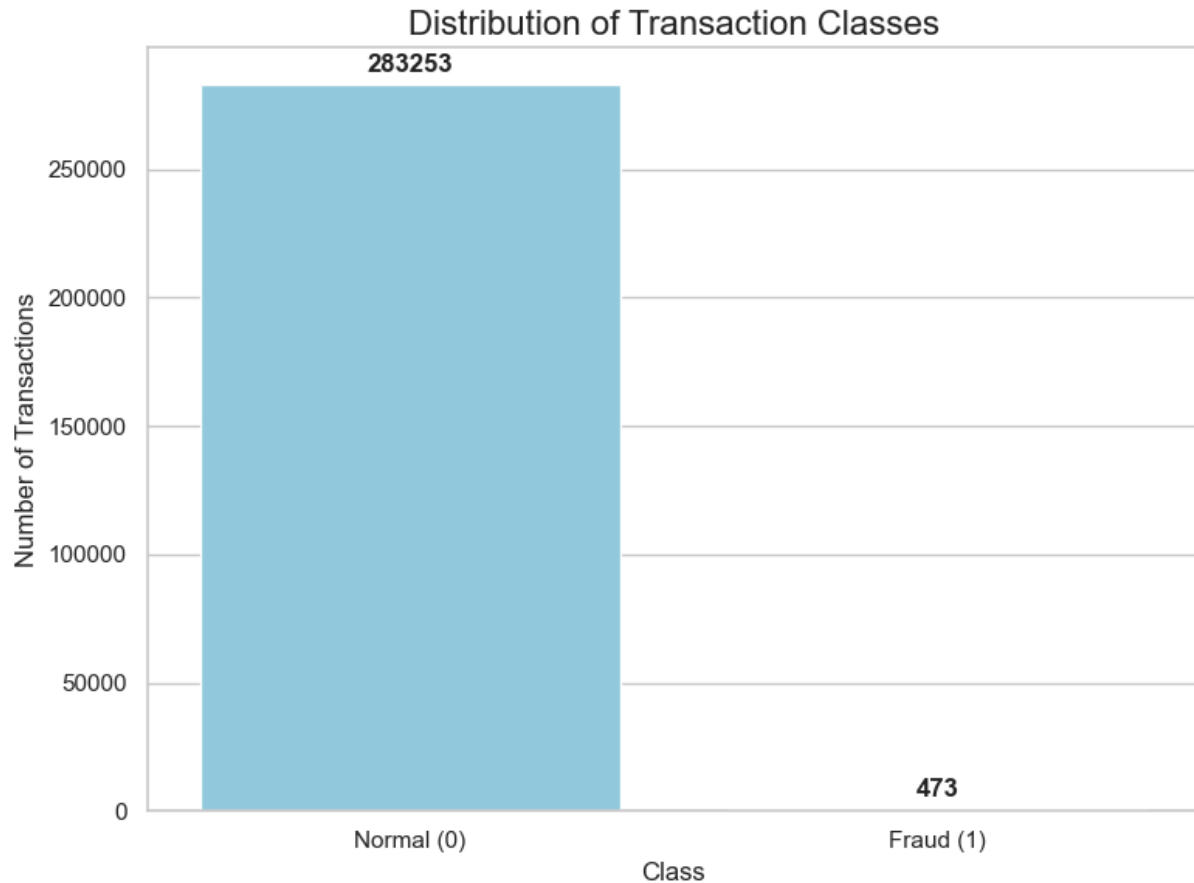
Data Cleaning

- There were **no missing values**, which is ideal.
- **1,081 duplicate rows** were found and removed, leaving **283,726 unique transactions**.
- All features were numerical:
 - Most were **float values**
 - The target column (**Class**) was an integer:
 - `0 = Normal transaction`
 - `1 = Fraudulent transaction`

3. Exploratory Data Analysis (EDA)

Class Imbalance

- Only **473 transactions (0.167%)** were fraudulent
- This means the dataset is **highly imbalanced**

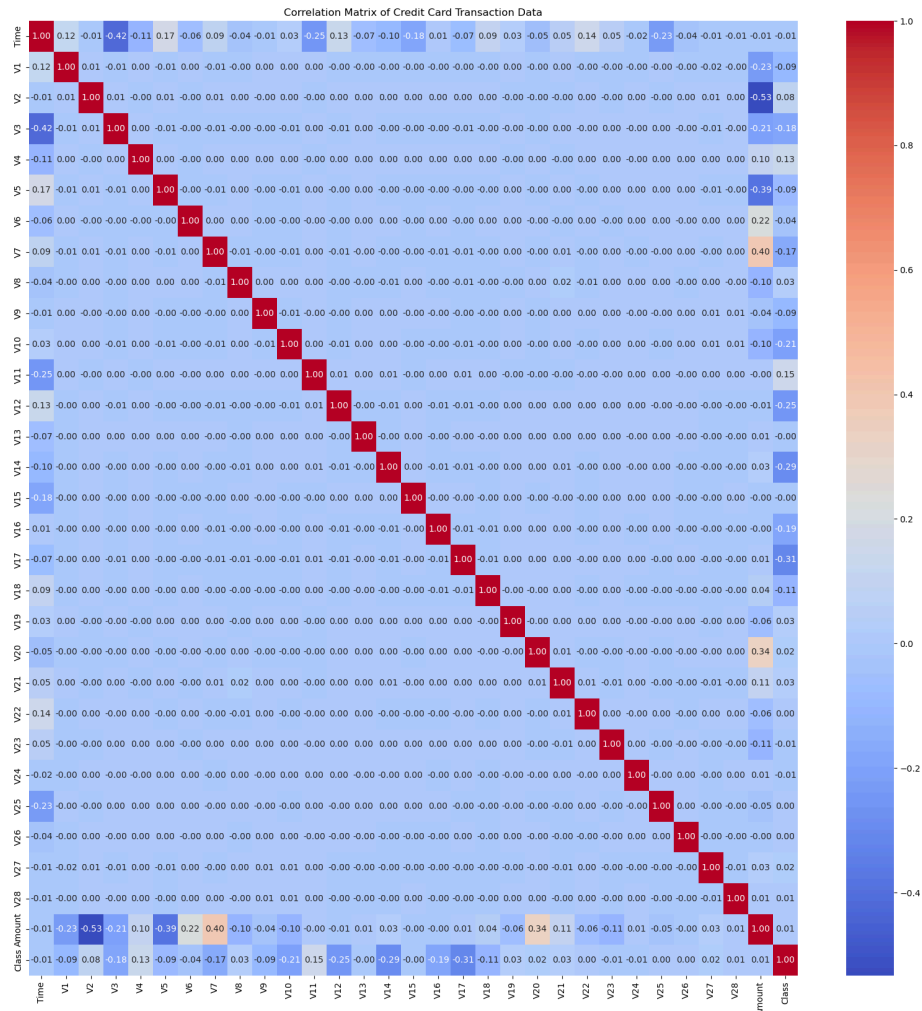


Why this matters:

If not handled properly, a model could predict “not fraud” every time and still be over 99% accurate, which is misleading.

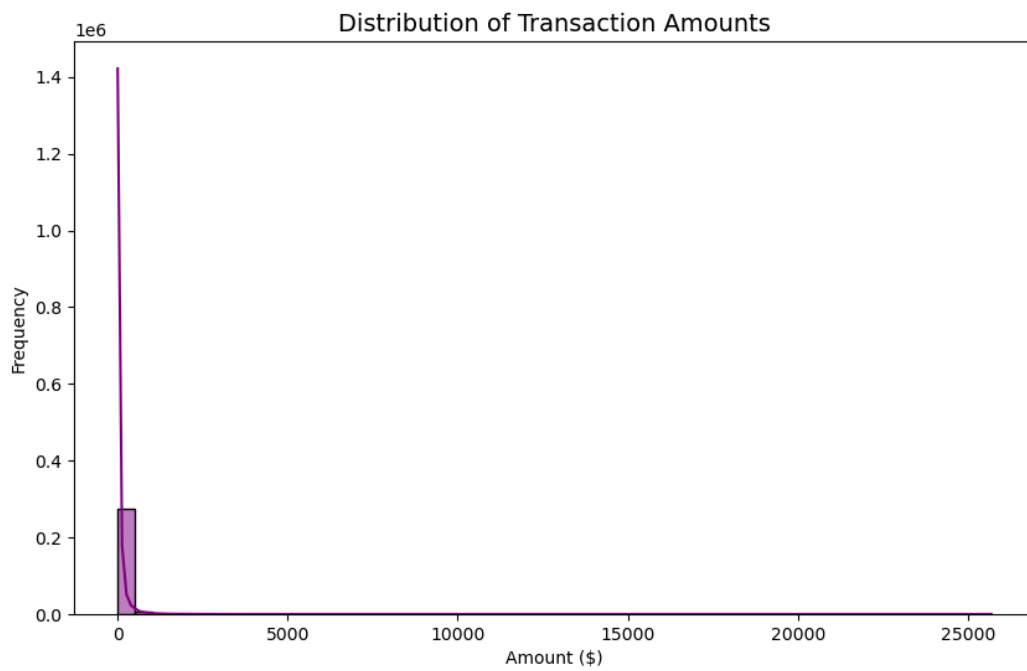
Feature Relationships

- A **correlation matrix** was created to observe relationships between features.
- However, feature selection was not based on correlation because:
 - Models like Random Forest and Gradient Boosting can automatically learn complex patterns, even from weakly correlated features.



Transaction Amount

- The distribution was **highly skewed (16.98)**
- This means most transactions are small, with a few very large ones.

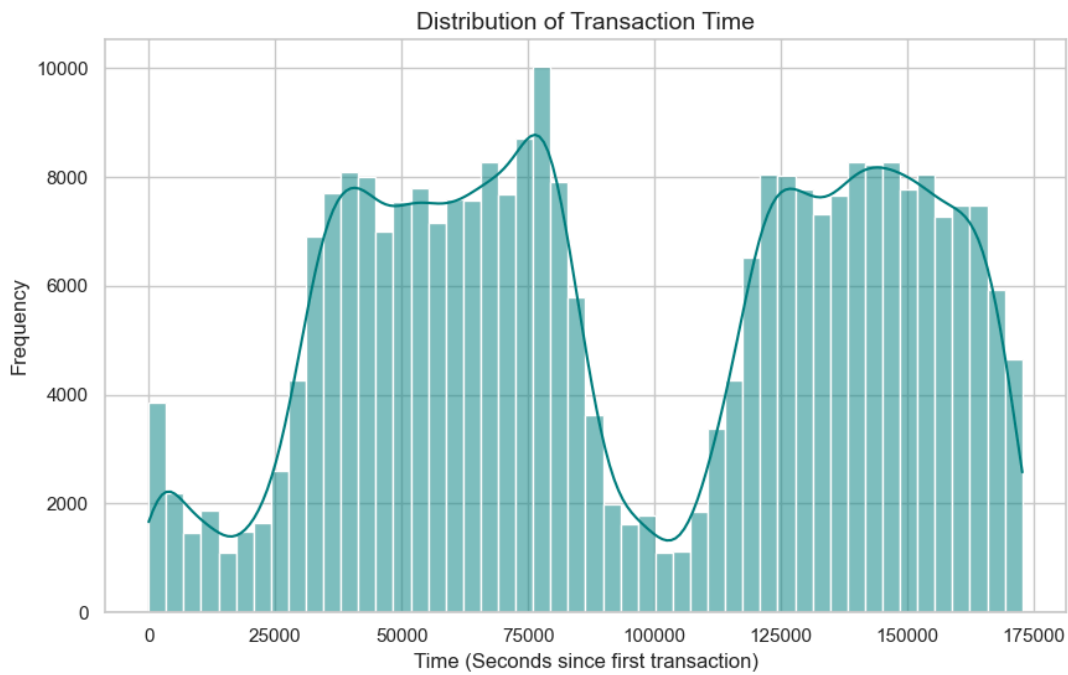


Action taken:

- **Scaling was applied** to normalize this feature.

Transaction Time

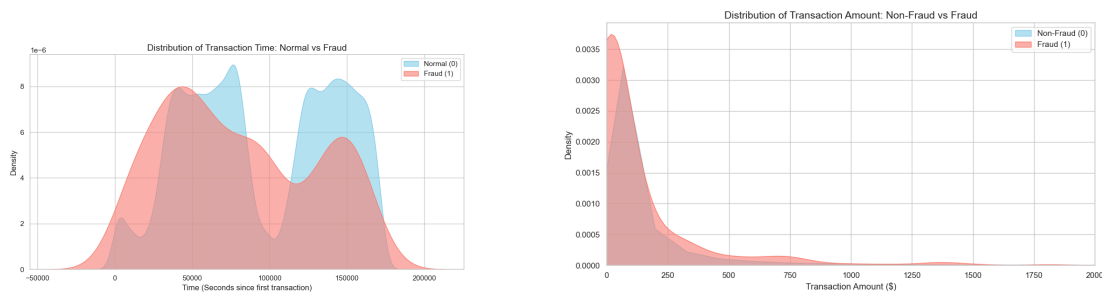
- Distribution was nearly normal (**skewness ≈ -0.04**)



- Scaling was still applied for consistency.

Fraud Patterns Observed

- Fraudulent transactions:
 - Occur **less frequently after 50,000 seconds**
 - Are slightly more common in **small amounts (\$50–\$100)**



4. Data Preprocessing

Train-Test Split

- Data was split into:
 - **80% training**

- **20% testing**
- **Stratified sampling** was used to preserve the fraud ratio.

Feature Scaling

- Applied **StandardScaler**
- Saved as `scaler.pkl` for future deployment (e.g., FastAPI)

5. Handling Class Imbalance

To address imbalance, two techniques were used:

1. Random Undersampling

- Reduces the number of normal transactions
- Makes dataset balanced by removing data

Advantage:

- Faster training

Disadvantage:

- Loss of useful information

2. SMOTE (Synthetic Minority Oversampling Technique)

- Generates synthetic fraud examples

Advantage:

- Keeps all original data

Disadvantage:

- Can introduce noise or unrealistic patterns

6. Model Training

Three models were trained on each dataset version:

- Logistic Regression
- Random Forest
- Gradient Boosting

Each model was tested on:

- Original dataset
- Undersampled dataset
- SMOTE dataset

7. Model Performance Comparison

Summary Table

Model	Dataset	Precision	Recall	F1 Score	ROC-AUC	PR-AUC
Random Forest	SMOTE	0.9114	0.7579	0.8276	0.9656	0.8038
Random Forest	Original	0.9718	0.7263	0.8313	0.9239	0.7876
Gradient Boosting	SMOTE	0.1149	0.8421	0.2023	0.9771	0.7358
Random Forest	Undersampled	0.0848	0.8737	0.1546	0.9767	0.6983
Logistic Regression	SMOTE	0.0530	0.8737	0.1000	0.9619	0.6769
Logistic Regression	Original	0.0562	0.8737	0.1057	0.9658	0.6719
Gradient Boosting	Original	0.8611	0.6526	0.7425	0.8539	0.6234
Logistic Regression	Undersampled	0.0519	0.8737	0.0980	0.9559	0.6182
Gradient Boosting	Undersampled	0.0442	0.8632	0.0841	0.9725	0.5754

8. Key Insights

Precision vs Recall Trade-off

- **Precision:** How many flagged fraud cases are actually fraud
- **Recall:** How many actual fraud cases were detected

Important trade-off:

- High recall increases fraud detection but also increases false alarms
- High precision reduces false alarms but may miss fraud cases

Model Observations

1. Logistic Regression

- High recall but very low precision
- Flags too many normal transactions as fraud

Not ideal for real-world use

2. Gradient Boosting

- Strong ROC-AUC scores
- Inconsistent performance:
 - SMOTE version had high recall but very poor precision

Risk: Too many false positives

3. Random Forest

- Best overall balance
- Strong performance across datasets

Key strengths:

- High precision
- Good recall
- Best F1 scores

Impact of Imbalance Handling

Method	Effect
Original Data	Best precision, stable performance
Undersampling	High recall but very poor precision
SMOTE	Balanced but increases false positives

9. Best Model Selection

Selected Model: Random Forest (SMOTE Dataset)

Justification

In fraud detection, the primary objective is to **identify as many fraudulent transactions as possible**, even if it means increasing false alarms. This makes **Recall** and **ROC-AUC** more important than Precision.

- **Recall (0.7579):**
This model successfully identifies a high proportion of fraudulent transactions. Missing fraud (false negatives) is more costly than flagging legitimate transactions.
- **ROC-AUC (0.9656):**
Indicates excellent ability to distinguish between fraudulent and non-fraudulent transactions across different thresholds.
- **F1 Score (0.8276):**
Shows a strong balance between precision and recall.
- **Precision (0.9114):**
While not the highest, it remains strong, meaning most flagged transactions are still genuinely fraudulent.

Why Not Random Forest (Original Dataset)?

Although it has slightly higher precision (0.9718) and F1 score (0.8313), it has:

- **Lower Recall (0.7263)**
- Slightly lower ROC-AUC (0.9239)

This means it **misses more fraudulent transactions**, which is undesirable in a fraud detection system.

Why SMOTE Works Better Here

- SMOTE generates synthetic fraud examples, helping the model better learn fraud patterns.
- This improves the model's ability to detect fraud (higher recall and ROC-AUC).
- The trade-off is a small increase in false positives, which is acceptable in this domain.

Conclusion

Fraud detection systems must prioritize **minimizing missed fraud cases** over reducing false alarms. Based on this objective, the **Random Forest model trained on the SMOTE dataset** provides the best performance.

It achieves a strong balance while significantly improving the detection of fraudulent transactions, making it the most suitable model for deployment in real-world fraud detection scenarios.

Recommendation

1. Deploy the **Random Forest (SMOTE)** model for fraud detection
2. Accept a moderate increase in false positives to ensure fewer missed fraud cases
3. Continuously monitor recall and ROC-AUC as key performance metrics

4. Fine-tune classification thresholds to align with business risk tolerance
5. Combine with human review systems to handle flagged transactions efficiently